Online Hypergradient Meta-Learning With Hypergradient Distilation

Ignashin Igor

Bayesian multimodeling
Department of Intelligent Systems, MIPT

March 2025

Hyperparameter Differentiation

Notations:

- w — model weights,
- λ — hyperparameters.
- D_t - current minibatch

Weight update rule:

$$w_t = \Phi(w_{t-1}, \lambda; D_t)$$

where Φ is the update function.

Hyperparameter optimization:

$$\min_{\lambda} \mathcal{L}_{val}(w_T(\lambda), \lambda)$$

Gradient decomposition:

$$\frac{d\mathcal{L}_{val}}{d\lambda} = \underbrace{\frac{\partial \mathcal{L}_{val}}{\partial \lambda}}_{g_T^{FO}} + \underbrace{\frac{\partial \mathcal{L}_{val}}{\partial w_T} \cdot \frac{dw_T}{d\lambda}}_{g_T^{SO}}$$

- g_T^{FO} — first-order gradient, computed directly.
- g_T^{SO} — second-order gradient, accounts for indirect influence of λ.

What are some examples where $g_T^{FO} = 0$?

Weights implicit hypergradient

Hypergradient of w_T:

$$\frac{dw_T}{d\lambda} = \frac{\partial \Phi(w_{T-1}, \lambda; D_T)}{\partial \lambda} + \frac{\partial \Phi(w_{T-1}, \lambda; D_T)}{\partial w_{T-1}} \cdot \frac{dw_{T-1}}{d\lambda}$$

Expanded hypergradient form:

$$\frac{dw_T}{d\lambda} = \sum_{t=1}^{T} \left(\prod_{s=t+1}^{T} A_s \right) B_t$$

where:

$$A_s = \frac{\partial \Phi(w_{s-1}, \lambda; D_s)}{\partial w_{s-1}}, \quad B_t = \frac{\partial \Phi(w_{t-1}, \lambda; D_t)}{\partial \lambda}$$

Key challenge: high computational cost (FMD, RMD).
Goal: approximate the equation for efficient optimization.

Reverse-Mode Differentiation (RMD)

Input: The last weight w_T and all previous weights $w_0, \ldots, w_{T-1}$.
Output: Hypergradient $g^{FO} + g^{SO}$.

Algorithm 1 RMD

$\alpha \leftarrow \frac{\partial \mathcal{L}_{val}(w_T, \lambda)}{\partial w_T}$
$g^{FO} \leftarrow \frac{\partial \mathcal{L}_{val}(w_T, \lambda)}{\partial \lambda}$
$g^{SO} \leftarrow 0$
for $t = T$ ***downto*** 1 **do**
  $g^{SO} \leftarrow g^{SO} + \alpha B_t$
  $\alpha \leftarrow \alpha A_t$
return $g^{FO} + g^{SO}$

Key Points:

- **Fast:** Computes only 1–2 Jacobian-Vector Products (JVPs) per step.
- **Memory-intensive:** Requires storing all previous weights $w_0, \ldots, w_{T-1}$.
- **Effective for short horizons** (e.g., few-shot learning, $T = 5$).

Trajectory Approximation

Instead of storing all weights, the training trajectory can be approximated using linear interpolation between the initial w_0 and final w_T.

DrMAD Algorithm (Fu et al., 2016):

$$\hat{w}_t = (1 - \frac{t}{T})w_0 + \frac{t}{T}w_T, \quad t = 1, \ldots, T-1$$

Each intermediate weight $\hat{w}_t$ is approximated as a linear combination of w_0 and w_T.

Approximate Jacobian Matrices:

$$\hat{A}_s = \frac{\partial \Phi(\hat{w}_{s-1}, \lambda; D_s)}{\partial w_{s-1}}, \quad \hat{B}_t = \frac{\partial \Phi(\hat{w}_{t-1}, \lambda; D_t)}{\partial \lambda}$$

Advantages:

- Reduces memory complexity by avoiding storage of the entire trajectory.

Disadvantages:

- The total number of computations grows quadratically:

$$\sum_{t=1}^{T}(2t-1) = T^2$$

  making DrMAD non-scalable for online optimization.

Short-Horizon Approximation

Main Idea: (Luketina et al., 2016)
Instead of computing the full trajectory, only the last step is analyzed. The previous weight w_{t-1} is treated as a constant, simplifying the gradient w.r.t. the hyperparameter:

$$\frac{dw_t}{d\lambda} \approx \left.\frac{\partial w_t}{\partial \lambda}\right|_{w_{t-1}} = B_t$$

Alternative Approximation: (Flennerhag et al., 2019; Ryu et al., 2020.)
Ignore second-order derivatives, approximating:

$$\frac{dw_t}{d\lambda} \approx 0$$

Advantages:

- Significantly reduces computational cost.
- Enables online hyperparameter optimization in high-dimensional problems.

Disadvantages:

- Susceptible to short-horizon bias (Wu et al., 2018).

Hypergradient Distillation

Key Idea: Distilling the second-order gradient g_t^{SO} into a **single Jacobian-vector product (JVP)** computed at the **distilled weight point** w and on the **distilled dataset** D. The normalized JVP is:

$$f_t(w, D) := \sigma\left(\alpha_t \frac{\partial \Phi(w, \lambda; D)}{\partial \lambda}\right)$$

where $\sigma(x) = \frac{x}{\|x\|_2}$.
The distillation task for each step of online hyperparameter optimization $t = 1, \ldots, T$ is:

$$\pi_t^*, w_t^*, D_t^* = \arg\min_{\pi, w, D} \|\pi f_t(w, D) - g_t^{SO}\|_2^2$$

Instead of g_t^{SO}, use $\pi_t^* f_t(w_t^*, D_t^*)$. Now, only **one JVP** $f_t(w_t^*, D_t^*)$ needs to be computed per step, compared to $2t - 1$ JVPs in RMD or DrMAD.
Advantage: This distillation helps **remove short-horizon bias** by encoding the entire trajectory into the JVP.

Distillation Solution

Notice that solving only w.r.t. π simply a vector projection:

$$\tilde{\pi}_t(w, D) = f_t(w, D)^T g_t^{SO}$$

Substitute this into equation resulting in the optimization task:

$$w_t^*, D_t^* = \arg\max_{w,D} \tilde{\pi}_t(w, D)$$

Thus, w_t^* and D_t^* correspond to the direction of the hypergradient, and π_t^* corresponds to its magnitude.
Solving equation requires computing g_t^{SO} for all $t = 1, \ldots, T$, which is the quantity we aim to approximate.
The task is to approximately solve equation without explicitly computing g_t^{SO}.

Hypergradient Direction Distillation: Hessian Approximation

We begin by simplifying the optimization objective $\tilde{\pi}_t(w, D) = f_t(w, D)^T g_t^{SO}$. The second-order gradient g_t^{SO} is approximated as:

$$g_t^{SO} = \alpha_t \sum_{i=1}^{t} \left(\prod_{j=i+1}^{t} A_j \right) B_i \approx \sum_{i=1}^{t} \gamma^{t-i} \alpha_t B_i$$

where $\gamma \geq 0$ is tuned on a meta-validation set. For online optimization, further distillation is needed.

The approximation of the Hessian for standard SGD with learning rate η_{Inner} is given by:

$$A_j = \nabla_{w_{j-1}} \left(w_{j-1} - \eta_{\text{Inner}} \nabla L_{\text{train}}(w_{j-1}, \lambda) \right)$$

We approximate the Hessian as $\nabla^2_{w_{j-1}} L_{\text{train}}(w_{j-1}, \lambda) \approx kI$, yielding the update rule:

$$A_j = I - \eta_{\text{Inner}} \cdot kI \approx (1 - \eta_{\text{Inner}} k) I = \gamma I$$

Substituting this into equation gives:

$$\tilde{\pi}_t(w, D) \approx \hat{\pi}_t(w, D) = \sum_{i=1}^{t} \delta_{t,i} \cdot f_t(w, D)^T f_t(w_{i-1}, D_i)$$

where $\delta_{t,i} = \gamma^{t-i} \|\alpha_t B_i\|_2 \geq 0$.

Lipschitz Assumption and Sequential Update

The maximum of $\hat{\pi}_t$ is achieved when $f_t(w, D)$ aligns well with the previous $f_t(w_0, D_1), \ldots, f_t(w_{t-1}, D_t)$. This is captured by the Lipschitz continuity assumption:

$$\|f_t(w, D) - f_t(w_{i-1}, D_i)\|_2 \leq K\|(w, D) - (w_{i-1}, D_i)\|_X$$

For the X-metric, we define:

$$K^2\|(w, D)\|_X^2 = K_1^2\|w\|_2^2 + K_2^2\|D\|_2^2$$

Taking the square of both sides and summing over all $i = 1, \ldots, t$, we obtain a lower bound for $\hat{\pi}_t$:

$$2\sum_{i=1}^{t}\delta_{t,i} - K_1^2\sum_{i=1}^{t}\delta_{t,i}\|w - w_{i-1}\|_2^2 - K_2^2\sum_{i=1}^{t}\delta_{t,i}\|D - D_i\|_2^2 \leq \hat{\pi}_t(w, D)$$

Instead of directly maximizing $\hat{\pi}_t$, we now maximize this lower bound, which leads to the following minimization problems:

$$\min_w \sum_{i=1}^{t}\delta_{t,i}\|w - w_{i-1}\|_2^2, \quad \min_D \sum_{i=1}^{t}\delta_{t,i}\|D - D_i\|_2^2$$

Efficient Sequential Update

Preview optimization problems describes how to determine the distilled w_t^* and D_t^* for each HO step $t = 1, \ldots, T$.
Since directly computing $\|\alpha_t B_i\|_2$ is expensive, we approximate it as $\|\alpha_t B_0\|_2 \approx \|\alpha_t B_1\|_2 \approx \cdots \approx \|\alpha_t B_t\|_2$, leading to the following weighted average as the approximated solution for w_t^*:

$$w_t^* \approx \frac{\gamma^{t-1}}{\sum_{i=1}^{t} \gamma^{t-i}} w_0 + \frac{\gamma^{t-2}}{\sum_{i=1}^{t} \gamma^{t-i}} w_1 + \cdots + \frac{\gamma^{0}}{\sum_{i=1}^{t} \gamma^{t-i}} w_{t-1}$$

To efficiently evaluate this expression for each HO step, we introduce the sequential update rule. Defining $p_t = \frac{\gamma - \gamma^t}{1-\gamma^t} \in [0, 1)$, we obtain:

$$t = 1 : w_1^* \leftarrow w_0,$$
$$t \geq 2 : w_t^* \leftarrow p_t w_{t-1}^* + (1 - p_t) w_{t-1}$$

This update rule eliminates the need to explicitly evaluate g_t^{SO}, reducing computational complexity while incorporating past learning trajectories $w_0, w_1, \ldots, w_{t-1}$ through a weighted running average.

Dataset Update and Role of γ

To efficiently update the dataset, we assume an Euclidean distance metric similar to w. However, defining Euclidean distance between datasets is non-trivial. Instead, we interpret p_t and $1 - p_t$ as probabilities for subsampling each dataset:

$$t = 1 : D_1^* \leftarrow D_1,$$
$$t \geq 2 : D_t^* \leftarrow SS(D_{t-1}^*, p_t) \cup SS(D_t, 1 - p_t)$$

where $SS(D, p)$ denotes random subsampling of approximately $|D|p$ instances from D. While a better metric for datasets may exist, we leave this as future work.

Role of γ: The parameter γ controls how much past information is retained. Larger values incorporate longer learning trajectories, while reduces to a simple one-step lookahead ignoring past steps.

Linear Estimator for π_t^*

To efficiently estimate π_t^* without full evaluation, we use a linear function $c_\gamma(t;\theta)$.
This function periodically fits $\theta \in \mathbb{R}$, the parameter of the estimator, allowing its use instead of directly computing π_t^* at each HO step.
Observing that the lower bound is approximately proportional to $\sum_{i=1}^{t} \delta_{t,i} = \sum_{i=1}^{t} \gamma^{t-i} \|\alpha_t B_i\|_2$, we define the estimator as:

$$c_\gamma(t;\theta) = \theta \cdot \|v_t\|_2 \cdot \sum_{i=1}^{t} \gamma^{t-i} \frac{\partial \Phi(w_t^*, \lambda; D_t^*)}{\partial \lambda},$$

where $v_t := \alpha_t \frac{\partial \Phi(w_t^*, \lambda; D_t^*)}{\partial \lambda}$.
This linear estimator provides a computationally efficient alternative to direct evaluation, leveraging past trajectory information while maintaining a manageable computational cost.

Collecting Samples and Estimating θ

We collect samples $\{(x_s, y_s)\}_{s=1}^{T}$, where:

$$x_s = \|v_s\|_2 \cdot \frac{1-\gamma^s}{1-\gamma}, \quad y_s = \sigma(v_s)^T g_s^{SO}.$$

To compute terms efficiently:
1. Second-order term g_s^{SO}: Backpropagate g^{SO} from $t = T$ to $t = 1$ using DrMAD for all s in a single pass.
2. Distilled JVP v_s: Compute distilled parameters sequentially:

$$s = 1 : w_1^* \leftarrow w_{T-1}, \quad s \geq 2 : w_s^* \leftarrow p_s w_{s-1}^* + (1-p_s) w_{T-s},$$
$$s = 1 : D_1^* \leftarrow D_T, \quad s \geq 2 : D_s^* \leftarrow \mathrm{SS}(D_{s-1}^*, p_s) \cup \mathrm{SS}(D_{T-s+1}, 1-p_s).$$

Then, $v_s = \alpha_{s+(T-s)} \frac{\partial \Phi(w_s^*, \lambda; D_s^*)}{\partial \lambda}$.
Finally, estimate θ using least squares:

$$\theta = \frac{x^T y}{x^T x}, \quad \text{where } x = (x_1, \ldots, x_T), \quad y = (y_1, \ldots, y_T).$$

Estimation is performed periodically (e.g., every 50 inner optimizations).

Hyper Distil Algorithm

Algorithm 3 HyperDistill

Input: $\gamma \in [0, 1]$, initial λ, and initial ϕ.
Output: Learned hyperparameter λ.
for $m = 1$ **to** M **do**
  if $m \in$ `EstimationPeriod` **then**
    $\theta \leftarrow$ `LinearEstimation`(γ, λ, ϕ)
  end if
  $w_0 \leftarrow \phi$
  for $t = 1$ **to** T **do**
    $w_t^*, D_t^* \leftarrow$ Eq. (13), Eq. (14).
    $\pi_t^* \leftarrow c_\gamma(t; \theta)$ in Eq. (15)
    $w_t \leftarrow \Phi(w_{t-1}, \lambda; D_t)$
    $g \leftarrow g_t^{\text{FO}} + \pi_t^* f_t(w_t^*, D_t^*)$
    $\lambda \leftarrow \lambda - \eta^{\text{Hyper}} g$
  end for
  $\phi \leftarrow \phi - \eta^{\text{Reptile}}(\phi - w_T)$
end for

Algorithm 4 `LinearEstimation`(γ, λ, ϕ)

Input: $w_0 \leftarrow \phi$
for $t = 1$ **to** T **do**
  $w_t \leftarrow \Phi(w_{t-1}, \lambda; D_t)$
end for
$\alpha, \alpha_T \leftarrow \frac{\partial \mathcal{L}^{\text{val}}(w_T, \lambda)}{\partial w_T}, \quad g^{\text{SO}} \leftarrow 0$
for $t = T$ **downto** 1 **do**
  $\hat{w}_{t-1} \leftarrow \left(1 - \frac{t-1}{T}\right) w_0 + \frac{t-1}{T} w_T$
  $g^{\text{SO}} \leftarrow g^{\text{SO}} + \alpha \hat{B}_t$ (Eq. (16))
  $\alpha \leftarrow \alpha \hat{A}_t$
  $s \leftarrow T - t + 1$
  $w_s^*, D_s^* \leftarrow$ Eq. (17), Eq. (18)
  $v_s \leftarrow \alpha_T \frac{\partial \Phi(w_s^*, \lambda; D_s^*)}{\partial \lambda}$
  $x_s \leftarrow \|v_s\|_2 \cdot \frac{1-\gamma^s}{1-\gamma}, \quad y_s \leftarrow \sigma(v_s)^\top g^{\text{SO}}$
end for
return $(x^\top y)/(x^\top x)$

Conclusion

- HyperDistill outperforms offline methods like DrMAD, offering faster meta-convergence and better generalization.
- It effectively approximates the second-order term, alleviating the short horizon bias.
- HyperDistill achieves competitive performance while being computationally efficient, requiring a comparable number of JVPs per inner optimization.